Lecture 6: Functions

INFO 1100: Introduction to Programming

Functions

Functions

What is a function?

Functions

A piece of code that
does a task

(W3 Schools)

Functions

Modular units of code
designed to perform
specific tasks.

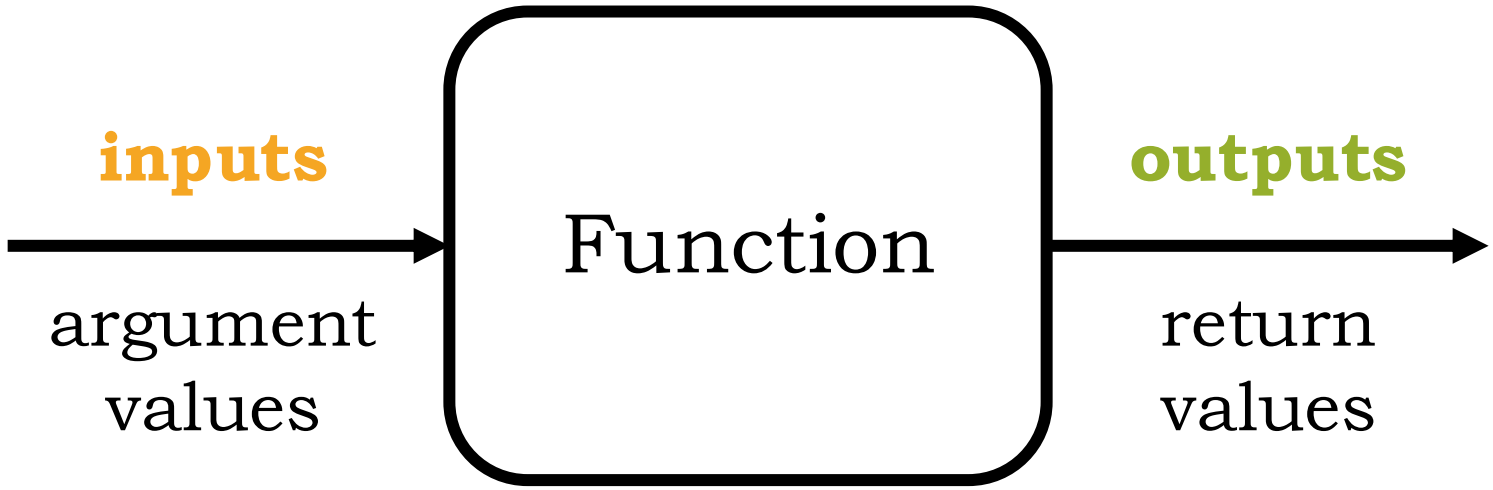
(GeeksforGeeks)

Functions

Value transformers!

(CS 1110)

Functions



Why functions?

Why functions?

Reuse code
without
re-writing it

Why functions?

Reuse code without re-writing it

Group conceptually related code together

Why functions?

Reuse code without re-writing it

Only make changes once

Group conceptually related code together

Why functions?

Reuse code without re-writing it

Debug easier

Only make changes once

Group conceptually related code together

We've used functions before!

We've used functions before!

Examples?

Defining functions

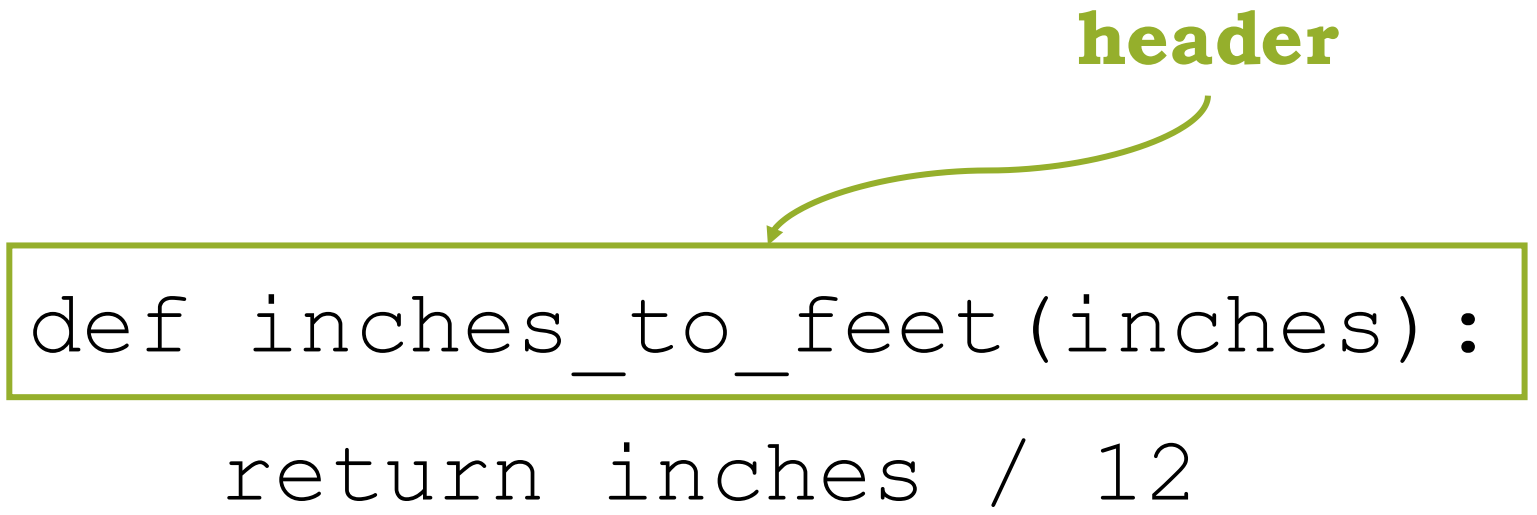
Enter in the chat:

What's a function you want to write?

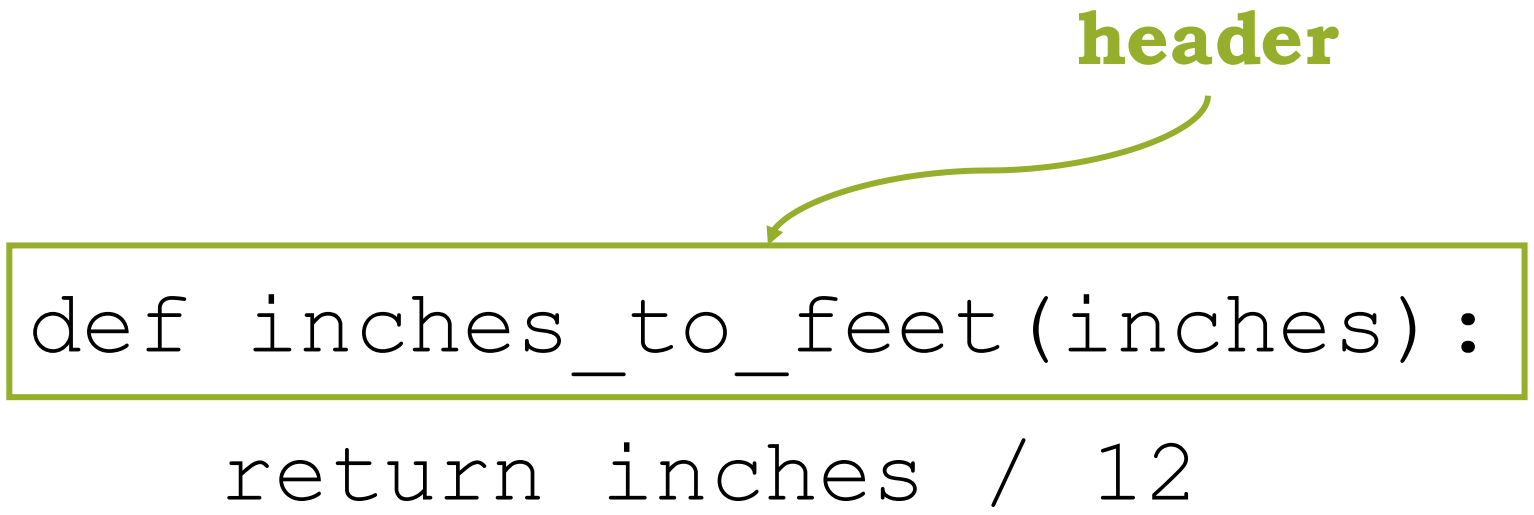
Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

Defining functions

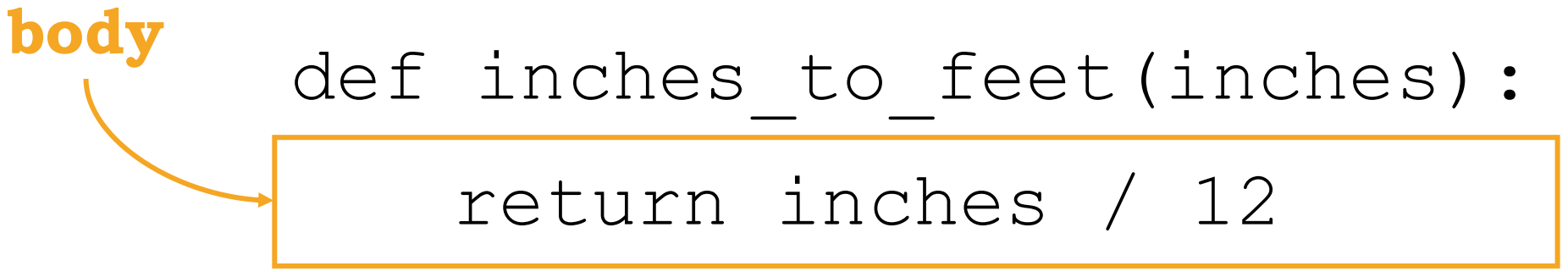


Defining functions

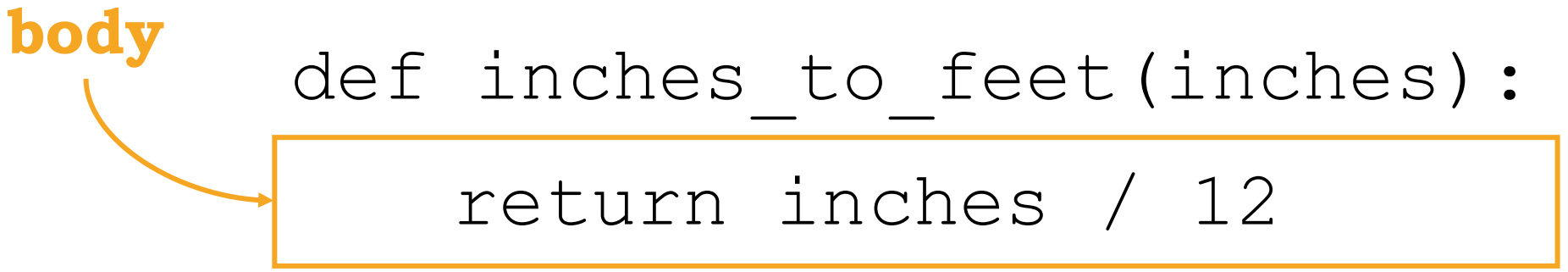


header: starts with `def` and ends with a colon

Defining functions



Defining functions



body: code indented underneath the header – statements and expressions

Defining functions

Defining functions

name: used to call function!
should be informative, like
variable names

Defining functions

parameter

```
def inches_to_feet(inches):
    return inches / 12
```

Defining functions

parameter

```
def inches_to_feet(inches):
    return inches / 12
```

parameter(s): variable(s) named in the header that can be used in the body. represent input values given to the function when it is called.

Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

return: determines the output function of each function. function execution ends with return.

Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

return moves data around in memory

print only makes symbols appear on screen

Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

if there is no **return**, function execution ends with the last line and it returns **None**

Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

code after the **return** is never run!

Defining functions

```
def inches_to_feet(inches):
    return inches / 12
```

```
def inches_to_feet(inches):
    print(inches / 12)
```

Defining functions

```
def inches_to_feet(inches):
    return inches / 12

print(inches_to_feet(36))
```



```
def inches_to_feet(inches):
    print(inches / 12)

print(inches_to_feet(36))
```

Defining functions

```
def inches_to_feet(inches):
    return inches / 12

print(inches_to_feet(36))
```

3

```
def inches_to_feet(inches):
    print(inches / 12)

print(inches_to_feet(36))
```

3

None

Defining functions

Function call

Function definition

Defining functions

Function call

```
inches_to_feet(65)
```

Function definition

```
def inches_to_feet(inches):
    return inches / 12
```

Defining functions

Function call

```
inches_to_feet(65)
```

- Tells Python to execute the function

Function definition

```
def inches_to_feet(inches):
    return inches / 12
```

- Tells Python what to do when the function is executed

Defining functions

Function call

```
inches_to_feet(65)
```

- Tells Python to execute the function
- **Arguments** are in parentheses

Function definition

```
def inches_to_feet(inches):
    return inches / 12
```

- Tells Python what to do when the function is executed
- **Parameters** are in parentheses

Global vs. local variables

Global vs. **local** variables

Global variables are defined outside of any function

Variables defined within a function are **local**

Global vs. **local** variables

Global variables are defined outside of any function

Can be used anywhere in the script

Variables defined within a function are **local**

Global vs. **local** variables

Global variables are defined outside of any function

Can be used anywhere in the script

Variables defined within a function are **local**

Exist only within the function

Global vs. local variables

Global variables are defined outside of any function

Can be used anywhere in the script

Variables defined within a function are **local**

Exist only within the function

Once the function is done executing, local variables no longer exist

Global vs. local variables

If a **local** variable has the same name as a **global** variable…

Global vs. local variables

If a **local** variable has the same name as a **global** variable…

both exist;

the **local** value will be used

Let’s work through some examples!

Example 1

```
def double_printer(x):
    print(x * 2)

def double_returner(x):
    return x * 2
```

Example 1

```
def double_printer(x):
    print(x * 2)

def double_returner(x):
    return x * 2
```

```
result = double_printer(5)
```

Example 1

```
def double_printer(x):
    print(x * 2)

def double_returner(x):
    return x * 2
```

```
result = double_returner(5)
```

Example 2

```
def absolute_value(n):
    if n < 0:
        return -n
    return n
    print("Done computing!")
```

Example 2

```
def absolute_value(n):
    if n < 0:
        return -n
    return n
    print("Done computing!")
```

```
result = absolute_value(8)
```

Example 2

```
def absolute_value(n):
    if n < 0:
        return -n
    return n
    print("Done computing!")

result = absolute_value(-8)
```

Example 3

```
def ticket_price(age):
    if age < 18:
        return 10
    elif age < 13:
        return 5
    else:
        return 15
```

Example 3

```
def ticket_price(age):
    if age < 18:
        return 10
    elif age < 13:
        return 5
    else:
        return 15
```

What input values would trigger each return?

Example 4

```
def increment():
    count = count + 1
    return count

count = 10
```

Example 4

```
def increment():
    count = count + 1
    return count

count = 10

print(increment())
print(count)
```

Questions?

(… I'll wait for 3)

For tomorrow:

(07/02 @ 11:30am)

- Lab 6
- Readings

  Think Python –

  9.1-9.6, 9.8-9.11
- GRQs